

Parallel PageRank

Submission for for CSC304 Course Project - Spring 2026

Ibrahim Habib
Suez Canal University
ebrahim2023@ci.suez.edu.eg

2026-05-09

This is my submission for the course project for the course Parallel Programming (CSC 304) at the Suez Canal University in the spring semester of 2026.

The app implements the PageRank algorithm (Brin & Page (1998)) parallelly using Python's multiprocessing library. There is a streamlit interface that allows you to run the algorithm on the Stanford SNAP (Leskovec & Krevl (2014)) web graphs and compare the results with the sequential implementation. The datasets are:

- **web-Google**: Nodes represent web pages and directed edges represent hyperlinks between them. Released in 2002 by Google as a part of Google Programming Contest. (Leskovec et al. (2009); Google (2002))
- **web-BerkStan**: Nodes represent pages from berkely.edu and stanford.edu domains and directed edges represent hyperlinks between them. (Leskovec et al. (2009))
- **web-Stanford**: Nodes represent pages from Stanford University (stanford.edu) and directed edges represent hyperlinks between them. (Leskovec et al. (2009))
- **web-NotreDame**: Nodes represent pages from University of Notre Dame (domain nd.edu) and directed edges represent hyperlinks between them. (Albert et al. (1999))

You can access the app on Hugging Face spaces or run it locally by following the instructions in the repository.

Important Links

- GitHub Repository: <https://github.com/ibrahimhabibeg/parallel-pagerank>
- Hugging Face Space: <https://huggingface.co/spaces/ibrahimhabibeg/parallel-pagerank>

Warning

On the Hugging Face spaces, the app runs on a 2-core CPU, so not all the features of the app are available.

The PageRank Algorithm

The PageRank algorithm (Brin & Page (1998)) is used for ranking nodes in a graph according to their importance. It is famous for being the algorithm that initiated Google's success.

The algorithm is implemented as a Markov chain.

Given a graph with n nodes, we represent the probability of being at each node at the timestep t as a vector $\mathbf{v}_t \in \mathbb{R}^n$. We initialize $\mathbf{v}_0$ as a uniform distribution over the nodes, i.e., all values are $\frac{1}{n}$. Using the transition matrix $\mathbf{M} \in \mathbb{R}^{n \times n}$ (which will be defined below), we compute the next state of $\mathbf{v}$ as follows

$$\mathbf{v}_{t+1} = \mathbf{M}\mathbf{v}_t$$

Regarding the transition matrix $\mathbf{M}$, M_{ij} is the probability of transitioning from node j to node i . We define the damping factor α (usually set to 0.85) as the probability of transitioning to a neighboring node, and $1 - \alpha$ as the probability of transitioning to any random node with equal probability. Given that the surfer will move to a neighboring node, all neighbors of node j will have equal probability of being chosen (because we are assuming a non-weighted graph). If there are no neighbors for node j , then the surfer will transition to any random node with equal probability. Hence, we can write M_{ij} as follows

$$\mathbf{M} = \alpha(\mathbf{A}\mathbf{D}^{-1} + \mathbf{E}) + (1 - \alpha)\frac{1}{n}\mathbf{J}_n$$

where $\mathbf{A}$ is the adjacency matrix of the graph, $\mathbf{D}$ is the diagonal matrix where D_{jj} is the out-degree of node j , $\mathbf{J}_n$ is the $n \times n$ matrix of all ones, and $\mathbf{E}$ is a matrix where $E_{ij} = \frac{1}{n}$ if node j has no neighbors and 0 otherwise.

Algorithm Implementation

n is usually quite large; thus, we cannot possibly store $\mathbf{M}$ in memory, e.g. for the web-Google graph $\mathbf{M}$ will need 2.8 TB of memory using 32-bit floats. To overcome this issue, we rewrite the update step of $\mathbf{v}$ as follows

$$\begin{aligned}\mathbf{v}_{t+1} &= \mathbf{M}\mathbf{v}_t \\ &= \alpha\mathbf{A}\mathbf{D}^{-1}\mathbf{v}_t + \alpha\mathbf{E}\mathbf{v}_t + (1 - \alpha)\frac{1}{n}\mathbf{J}_n\mathbf{v}_t \\ &= \alpha\mathbf{A}\mathbf{D}^{-1}\mathbf{v}_t + \alpha\frac{1}{n}(\mathbf{e} \cdot \mathbf{v}_t) + (1 - \alpha)\frac{1}{n}\end{aligned}$$

where $\mathbf{e}$ is an indicator vector of the nodes with no neighbors and broadcasting is used in the last step to add the two scalar values to each element of the vector.

$\mathbf{A}\mathbf{D}^{-1}$ is a sparse matrix; hence, we are able to store it in memory and multiply with it efficiently.

Regarding the parallel implementation, we parallelize the multiplication by splitting the matrix into blocks of rows and multiplying each block with $\mathbf{v}_t$ in a separate process. Finally, we merge all the results together.

We keep moving in the chain until the L2 norm of the difference between $\mathbf{v}_{t+1}$ and $\mathbf{v}_t$ is less than a certain threshold or we reach a maximum number of iterations.

Acknowledgments

Special thanks to the course instructor Dr. Mohamed Khamis and the TA Eng. Amro Medhat for their guidance throughout the course.

References

- Albert, R., Jeong, H., & Barabási, A.-L. (1999). Diameter of the World-Wide Web. *Nature*, 401(6749), 130–131. <https://doi.org/10.1038/43601>
- Brin, S., & Page, L. (1998). The Anatomy of a Large-Scale Hypertextual Web Search Engine. *Computer Networks*, 30, 107–117. <https://snap.stanford.edu/class/cs224w-readings/Brin98Anatomy.pdf>
- Google. (2002). *Google Programming Contest*.
- Leskovec, J., & Krevl, A. (2014). *SNAP Datasets: Stanford Large Network Dataset Collection*. <http://snap.stanford.edu/data>
- Leskovec, J., Lang, K. J., Dasgupta, A., & Mahoney, M. W. (2009). Community Structure in Large Networks: Natural Cluster Sizes and the Absence of Large Well-Defined Clusters. *Internet Mathematics*, 6(1), 29–123. <https://doi.org/10.1080/15427951.2009.10129177>